

Centre Marocain de Recherches Polytechnique et d'Innovation

Département / Service : Cybersécurité IA

PROJET DE STAGE PFA – ÉTÉ 2026

**Système de détection et d'alerte précoce
des cybermenaces ciblant les PME**

LIVRABLE DU DEUXIÈME JALON

**Construction d'un premier détecteur
d'URL de phishing**

Feature Engineering, automatisation des données
et régression logistique

Réalisé par :

ALLADO Kossi Richard
ADJEVI Kodjo Marius-Ben

Encadré par :

Pr. Youssef Bentaleb
Co-encadrante : Pr. Mehdi Ajana

Période du deuxième jalon

Du 1^{er} au 15 août 2026

Année universitaire 2025–2026

Table des matières

1	Introduction	1
2	Objectifs et démarche du Jalon 2	2
2.1	Objectifs	2
2.2	Chaîne de traitement	2
3	Données utilisées	3
3.1	Jeu de données étiqueté	3
3.2	Différence entre URLhaus et le jeu d'entraînement	3
4	Extraction des caractéristiques	4
4.1	Principe du Feature Engineering	4
4.2	Caractéristiques retenues	4
4.3	Extrait de l'implémentation	4
5	Automatisation de la collecte	6
5.1	Améliorations apportées	6
5.2	Détection d'un changement	6
5.3	Mise à jour et historique	6
6	Construction du modèle	7
6.1	Choix de la régression logistique	7
6.2	Séparation entraînement/test	7
6.3	Standardisation	7
6.4	Entraînement et sauvegarde	7
7	Évaluation et résultats	9
7.1	Métriques utilisées	9
7.2	Résultats obtenus	9
7.3	Matrice de confusion	9
7.4	Interprétation	10
8	Prédiction d'une nouvelle URL	11
9	Organisation et reproductibilité	12
9.0.1	Organisation du dépôt	12
9.1	Reproduction de l'expérience	13
10	Limites et perspectives	14
10.1	Limites actuelles	14
10.2	Préparation du Jalon 3	14
11	Conclusion	15
	Références	16
A	Valeurs détaillées de l'exécution	17

1. Introduction

Le projet de stage PFA intitulé « Système de détection et d’alerte précoce des cybermenaces ciblant les PME » vise à construire progressivement un prototype capable de collecter, détecter et signaler des menaces liées au phishing. Le premier jalon a permis de comprendre le phishing, de présenter les URL malveillantes et de développer un script de collecte depuis la plateforme publique URLhaus.

Le deuxième jalon transforme ces données brutes en informations exploitables par un algorithme d’apprentissage automatique. L’objectif principal consiste à construire un premier détecteur simple, compréhensible et reproductible. Pour cela, nous avons préparé un jeu de données étiqueté, extrait des caractéristiques numériques à partir des URL, entraîné une régression logistique et évalué ses performances sur un ensemble de test indépendant.

À la suite du retour de l’encadrant, une attention particulière a également été portée à deux améliorations :

- enrichir les URL au moyen d’une phase de *Feature Engineering* ;
- automatiser la collecte, le nettoyage, la mise à jour et l’historisation des données.

Ce rapport décrit la démarche, les choix réalisés, les résultats obtenus et les limites du premier modèle. Le code source associé est disponible dans le dépôt GitHub du projet : https://github.com/ALLAKORI/STAGE_PFA_CMRPI.

2. Objectifs et démarche du Jalon 2

2.1 Objectifs

Les objectifs opérationnels du deuxième jalon sont les suivants :

1. récupérer un jeu de données d'URL déjà étiquetées ;
2. transformer les URL en variables numériques pertinentes ;
3. séparer les données d'entraînement et de test ;
4. entraîner un modèle simple avec `scikit-learn` ;
5. mesurer les bonnes détections et les erreurs ;
6. sauvegarder le modèle afin de prédire de nouvelles URL ;
7. rendre la collecte de données plus robuste et reproductible.

2.2 Chaîne de traitement

La chaîne mise en place suit les étapes ci-dessous :

Collecte ou chargement des URL → nettoyage → extraction des caractéristiques → séparation entraînement/test → standardisation → entraînement → évaluation et sauvegarde

Cette organisation sépare les responsabilités entre plusieurs scripts. Le fichier `Download_urls.py` gère la collecte, le fichier `feature_engineering.py` transforme une URL, le fichier `train_model.py` entraîne le détecteur et le fichier `predict.py` applique le modèle sauvegardé à une nouvelle URL.

3. Données utilisées

3.1 Jeu de données étiqueté

L'entraînement s'appuie sur le jeu de données *PhiUSIIL Phishing URL Dataset*. Celui-ci contient des URL légitimes et des URL de phishing, accompagnées d'une étiquette de classe. Après préparation et extraction des variables utiles, le fichier `data/processed/dataset_features.csv` contient 235 795 observations et 22 colonnes : 21 caractéristiques et une étiquette.

TABLE 3.1 – Répartition des classes dans le jeu préparé

Classe	Signification	Nombre d'URL
0	Phishing	100 945
1	Légitime	134 850
Total		235 795

Les deux classes sont suffisamment représentées pour effectuer un premier apprentissage. La légère différence entre leurs effectifs est prise en compte lors de la séparation grâce à l'option `stratify=y`, qui conserve des proportions comparables dans les ensembles d'entraînement et de test.

3.2 Différence entre URLhaus et le jeu d'entraînement

URLhaus fournit des indicateurs réels et récents associés principalement à la distribution de logiciels malveillants. Cette source est utile pour alimenter et actualiser la base locale, mais elle ne fournit pas à elle seule un échantillon équilibré d'URL légitimes et de phishing adapté à l'apprentissage supervisé.

Le jeu PhiUSIIL est donc utilisé pour entraîner et évaluer le premier modèle, tandis qu'URLhaus reste la source de Threat Intelligence destinée à la collecte continue. Cette distinction évite de considérer toutes les données comme interchangeables et clarifie le rôle de chaque source.

4. Extraction des caractéristiques

4.1 Principe du Feature Engineering

Un modèle de régression logistique ne peut pas interpréter directement une chaîne de caractères telle qu'une URL. Le *Feature Engineering* consiste à transformer cette chaîne en valeurs numériques décrivant sa structure. L'objectif n'est pas d'ouvrir le site, mais d'analyser uniquement l'adresse.

Cette analyse statique réduit le risque lié à la manipulation d'URL malveillantes et permet une prédiction rapide. Cependant, elle ne renseigne pas sur le contenu réel de la page, son certificat, son ancienneté ou son comportement après redirection.

4.2 Caractéristiques retenues

TABLE 4.1 – Les 21 caractéristiques extraites d'une URL

Caractéristique	Description
url_length	Longueur totale de l'URL.
domain_length	Longueur du nom d'hôte.
path_length	Longueur du chemin après le domaine.
query_length	Longueur de la chaîne de paramètres.
number_of_subdomains	Nombre approximatif de sous-domaines.
has_ip_address	Indique si une adresse IP remplace le domaine.
uses_https	Indique la présence du protocole HTTPS.
number_of_letters	Nombre de lettres.
number_of_digits	Nombre de chiffres.
number_of_special_characters	Nombre de caractères non alphanumériques.
letter_ratio	Proportion de lettres dans l'URL.
digit_ratio	Proportion de chiffres dans l'URL.
number_of_dots	Nombre de points.
number_of_hyphens	Nombre de tirets.
number_of_underscores	Nombre de tirets bas.
number_of_slashes	Nombre de barres obliques.
number_of_question_marks	Nombre de points d'interrogation.
number_of_equal_signs	Nombre de signes égal.
number_of_ampersands	Nombre d'esperluettes.
number_of_at_symbols	Nombre d'arobases.
suspicious_keyword_count	Nombre de mots suspects tels que login, verify, account ou password.

4.3 Extrait de l'implémentation

```
1 def extract_features(url):
2     original_url = str(url).strip()
3     normalized_url = normalize_url(original_url)
4     parsed_url = urlparse(normalized_url)
5
6     hostname = parsed_url.hostname or ""
7     path = parsed_url.path or ""
8     query = parsed_url.query or ""
9
```

```
10     return {
11         "url_length": len(original_url),
12         "domain_length": len(hostname),
13         "path_length": len(path),
14         "query_length": len(query),
15         "number_of_subdomains": count_subdomains(hostname),
16         "has_ip_address": contains_ip_address(hostname),
17         "uses_https": int(
18             original_url.lower().startswith("https://")
19         ),
20         # autres caracteristiques...
21     }
```

Listing 4.1 – Extrait de la fonction d'extraction

La fonction normalise d'abord l'adresse en ajoutant un protocole lorsqu'il est absent. La fonction `urlparse()` sépare ensuite le domaine, le chemin et les paramètres. L'adresse IP est vérifiée avec le module standard `ipaddress`, sans effectuer de connexion vers l'URL.

5. Automatisation de la collecte

5.1 Améliorations apportées

Le premier script téléchargeait les données URLhaus et remplaçait le fichier local. Pour le deuxième jalon, le processus a été enrichi afin de :

- vérifier les erreurs HTTP et limiter l'attente à 30 secondes ;
- nettoyer les commentaires présents dans le CSV d'URLhaus ;
- normaliser les URL et supprimer les doublons ;
- comparer les nouvelles données à la base locale ;
- ajouter uniquement les URL encore inconnues ;
- conserver un historique daté des nouvelles collectes ;
- calculer un hash SHA-256 du contenu téléchargé ;
- éviter un retraitement lorsque la source n'a pas changé ;
- mémoriser la date de la dernière mise à jour.

5.2 Détection d'un changement

```
1 contenu_brut = response.text
2
3 hash_actuel = hashlib.sha256(
4     contenu_brut.encode("utf-8")
5 ).hexdigest()
6
7 etat = charger_etat()
8
9 if etat.get("hash") == hash_actuel:
10     print("Aucun changement detecte.")
11     return
```

Listing 5.1 – Vérification du contenu téléchargé

Le hash représente une empreinte du fichier. Si l'empreinte actuelle est identique à celle de la collecte précédente, le programme s'arrête sans réécrire inutilement les données.

5.3 Mise à jour et historique

Les URL déjà présentes sont réunies dans un ensemble, puis les nouvelles lignes sont sélectionnées par différence. Après fusion, les doublons sont supprimés. Un fichier daté est créé dans `data/collected/urlhaus/history` afin de conserver uniquement les ajouts de chaque exécution.

Cette organisation améliore la robustesse, la maintenabilité et la reproductibilité demandées par l'encadrant. Le script peut ensuite être lancé périodiquement à l'aide du Planificateur de tâches Windows ou d'une tâche `cron` sous Linux. La programmation automatique par le système d'exploitation sera intégrée et documentée lors de l'assemblage final.

6. Construction du modèle

6.1 Choix de la régression logistique

La régression logistique est un algorithme supervisé simple destiné à la classification. Malgré son nom, elle permet ici de choisir entre deux classes : phishing ou légitime. Elle calcule une probabilité à partir d'une combinaison pondérée des caractéristiques.

Ce modèle a été retenu parce qu'il est rapide, accessible, adapté à une première classification binaire et plus facile à expliquer qu'un modèle complexe. Il fournit également une méthode `predict_proba()` utile pour afficher un niveau de confiance.

6.2 Séparation entraînement/test

Le jeu de données est divisé en deux parties :

- 80 % des observations servent à entraîner le modèle ;
- 20 % servent uniquement à mesurer ses performances.

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X,  
3     y,  
4     test_size=0.2,  
5     random_state=42,  
6     stratify=y,  
7 )
```

Listing 6.1 – Séparation stratifiée des données

Le paramètre `random_state=42` rend la séparation reproductible. L'ensemble de test contient 47 159 URL : 20 189 de phishing et 26 970 légitimes. Ces URL ne sont pas utilisées pendant l'apprentissage.

6.3 Standardisation

Les variables n'ont pas toutes la même échelle. Par exemple, une longueur d'URL peut atteindre plusieurs dizaines ou centaines, alors que `uses_https` vaut uniquement 0 ou 1. La standardisation centre et réduit les variables pour faciliter l'optimisation du modèle.

```
1 scaler = StandardScaler()  
2  
3 X_train_scaled = scaler.fit_transform(X_train)  
4 X_test_scaled = scaler.transform(X_test)
```

Listing 6.2 – Standardisation sans fuite de données

Le scaler est ajusté uniquement sur les données d'entraînement. L'ensemble de test est transformé avec les paramètres déjà appris, ce qui évite de transmettre au modèle des informations provenant du test.

6.4 Entraînement et sauvegarde

```
1 model = LogisticRegression(  
2     max_iter=1000,  
3     random_state=42,  
4 )
```

```
5  
6 model.fit(X_train_scaled, y_train)  
7  
8 joblib.dump(model, "models/logistic_regression_model.pkl")  
9 joblib.dump(scaler, "models/scaler.pkl")
```

Listing 6.3 – Entraînement de la régression logistique

Le modèle et le scaler sont enregistrés avec `joblib`. Une nouvelle prédiction peut ainsi être réalisée sans recommencer l'entraînement.

7. Évaluation et résultats

7.1 Métriques utilisées

L'*accuracy* mesure la proportion totale de prédictions correctes. La précision indique, parmi les URL annoncées comme phishing, la proportion réellement associée au phishing. Le rappel indique la proportion d'URL de phishing effectivement détectées. Le F1-score fournit un équilibre entre précision et rappel.

Dans notre codage, la classe 0 correspond au phishing et la classe 1 à une URL légitime. Le script évalue explicitement la classe 0 pour les métriques principales, car la détection du phishing est l'objectif prioritaire. Le rapport de classification détaille ensuite les résultats pour les deux classes.

7.2 Résultats obtenus

99,32 % Accuracy	99,91 % Précision phishing	98,49 % Rappel phishing	99,20 % F1-score phishing
----------------------------	---	-----------------------------------	--

TABLE 7.1 – Rapport de classification sur l'ensemble de test

Classe	Précision	Rappel	F1-score	Effectif
Phishing (0)	1,00	0,98	0,99	20 189
Légitime (1)	0,99	1,00	0,99	26 970
Accuracy		0,99		47 159
Moyenne macro	0,99	0,99	0,99	47 159
Moyenne pondérée	0,99	0,99	0,99	47 159

7.3 Matrice de confusion

TABLE 7.2 – Matrice de confusion

		Classe prédite	
		Phishing (0)	Légitime (1)
Classe réelle	Phishing (0)	19 885	304
	Légitime (1)	18	26 952

Les lignes représentent les classes réelles et les colonnes les classes prédites.

Sur 47 159 URL de test :

- 19 885 URL de phishing sont correctement détectées ;
- 26 952 URL légitimes sont correctement reconnues ;
- 304 URL de phishing sont classées à tort comme légitimes ;
- 18 URL légitimes sont classées à tort comme phishing.

L'erreur la plus importante pour la sécurité correspond aux 304 URL de phishing classées comme légitimes, car elles pourraient ne pas déclencher d'alerte. Le taux global est élevé, mais le prototype ne doit donc pas être considéré comme une garantie absolue.

7.4 Interprétation

Les résultats montrent que les caractéristiques lexicales et structurelles permettent de distinguer efficacement les classes dans ce jeu de données. Cependant, une performance élevée sur un ensemble provenant de la même source ne prouve pas que le modèle conservera exactement la même efficacité sur des URL réelles, nouvelles ou issues d'une autre distribution.

Il sera donc nécessaire de tester le détecteur sur des données plus récentes, de vérifier l'absence de doublons entre les ensembles et de surveiller l'évolution des performances dans le temps.

8. Prédiction d'une nouvelle URL

Le script `predict.py` charge le modèle et le scaler enregistrés. Il demande une URL à l'utilisateur, extrait les 21 caractéristiques dans le même ordre que pendant l'entraînement, applique la standardisation, puis affiche la classe et la probabilité associée.

```
1 url = input("Entrez une URL a analyser : ").strip()
2
3 features = extract_features(url)
4 features_df = pd.DataFrame([features])
5 features_scaled = scaler.transform(features_df)
6
7 prediction = model.predict(features_scaled)[0]
8 probabilities = model.predict_proba(features_scaled)[0]
```

Listing 8.1 – Principe de la prédiction interactive

Si la prédiction vaut 1, le résultat affiché est « URL légitime ». Si elle vaut 0, le résultat est « URL phishing ». La confiance affichée correspond à la probabilité calculée pour la classe prédite.

Cette utilisation reste une aide à la décision. L'utilisateur ne doit pas ouvrir une URL suspecte pour confirmer le résultat. Une adresse classée légitime peut être malveillante, notamment si elle utilise une technique qui n'est pas représentée dans le jeu d'entraînement.

9. Organisation et reproductibilité

9.0.1 Organisation du dépôt

Le dépôt du projet est organisé de manière à séparer les différentes étapes de la chaîne de traitement, depuis la collecte des données jusqu'à l'entraînement du modèle et son utilisation. Cette organisation facilite la maintenance du projet ainsi que la reproductibilité des expériences.

```
1 project/
2 |--- data/
3 |   |--- raw/
4 |   |   '--- PhiUSIIL_Phishing_URL_Dataset.csv
5 |   |--- processed/
6 |   |   '--- dataset_features.csv
7 |   '--- collected/
8 |       '--- urlhaus/
9 |           |--- current_urls.csv
10 |           '--- history/
11 |
12 |--- docs/
13 |   |--- evaluation.md
14 |   '--- report/
15 |
16 |--- models/
17 |   |--- logistic_regression_model.pkl
18 |   '--- scaler.pkl
19 |
20 |--- Download_urls.py
21 |--- feature_engineering.py
22 |--- train_model.py
23 |--- predict.py
24 |--- prepare_phishing_url_features.ipynb
25 |--- requirements.txt
26 '--- README.md
```

Les principaux éléments du dépôt sont décrits ci-dessous :

- **data/raw/** : contient le jeu de données d'origine utilisé pour l'entraînement des modèles.
- **data/processed/** : regroupe les jeux de données après extraction des caractéristiques.
- **data/collected/urlhaus/** : stocke les URL téléchargées depuis URLhaus ainsi que l'historique des différentes collectes.
- **docs/** : rassemble la documentation technique, les résultats d'évaluation et les sources du rapport.
- **models/** : contient les modèles entraînés ainsi que les objets nécessaires à la phase de prédiction (standardisation, sérialisation).
- **Download_urls.py** : automatise le téléchargement et la mise à jour des URL provenant d'URLhaus.
- **feature_engineering.py** : extrait les 21 caractéristiques utilisées par les modèles de classification.
- **train_model.py** : réalise l'entraînement, l'évaluation et la sauvegarde des modèles.
- **predict.py** : permet d'effectuer la prédiction du caractère malveillant d'une URL à partir d'un modèle entraîné.
- **prepare_phishing_url_features.ipynb** : notebook utilisé pour l'exploration des données et l'extraction des caractéristiques.

- **requirements.txt** : liste des dépendances Python nécessaires à l'exécution du projet.
- **README.md** : présente les instructions d'installation, d'utilisation et l'organisation générale du dépôt.

9.1 Reproduction de l'expérience

```
1 python -m venv venv
2 .\venv\Scripts\Activate.ps1
3 pip install -r requirements.txt
4 python train_model.py
5 python predict.py
```

Listing 9.1 – Installation et exécution sous Windows

Le dépôt Git permet de conserver l'historique des modifications et de partager une version commune entre les deux stagiaires. L'environnement virtuel et les caches ne sont pas publiés. Les dépendances nécessaires sont déclarées dans **requirements.txt**.

10. Limites et perspectives

10.1 Limites actuelles

- Le modèle analyse la forme de l'URL, mais pas le contenu de la page.
- HTTPS ne garantit pas qu'un site est légitime.
- Les mots-clés suspects peuvent aussi apparaître dans une URL légitime.
- Les performances peuvent diminuer sur des données plus récentes.
- La confiance du modèle n'est pas une preuve de sécurité.
- URLhaus concerne plus largement les URL diffusant des logiciels malveillants et ne représente pas uniquement le phishing.
- Le prototype ne doit pas ouvrir automatiquement les URL collectées.

10.2 Préparation du Jalon 3

Le troisième jalon assemblera les éléments dans un prototype complet. Les travaux prévus sont :

1. créer un tableau de bord Streamlit ;
2. afficher les informations et le résultat de détection ;
3. envoyer une alerte par e-mail lorsqu'une menace est détectée ;
4. rédiger deux à trois fiches réflexes destinées aux PME ;
5. tester la chaîne de bout en bout ;
6. préparer la documentation et la présentation finale.

Une attention particulière sera portée à la protection des identifiants de messagerie. Ils ne devront pas être enregistrés dans le code ni publiés sur GitHub. Ils seront chargés à partir de variables d'environnement ou d'un fichier local exclu par `.gitignore`.

11. Conclusion

Ce deuxième jalon a permis de construire un premier détecteur d'URL de phishing fondé sur une régression logistique. Les URL ont été transformées en 21 caractéristiques numériques décrivant leur longueur, leur domaine, leurs caractères et certains motifs suspects.

Le jeu de 235 795 observations a été séparé en ensembles d'entraînement et de test de manière stratifiée. Sur les 47 159 observations réservées au test, le modèle obtient une accuracy de 99,32 % et un F1-score de 99,41 %. L'analyse de la matrice de confusion montre 322 erreurs, dont 304 URL de phishing classées comme légitimes.

Le processus de collecte URLhaus a également été amélioré avec la suppression des doublons, la détection des changements, la mise à jour incrémentale et la conservation d'un historique. Ces améliorations répondent aux recommandations formulées après le premier jalon.

Le modèle constitue une base fonctionnelle pour le prototype, mais il conserve des limites et ne doit pas être utilisé comme unique preuve de sécurité. Le prochain jalon intégrera ce détecteur dans une interface Streamlit avec un mécanisme d'alerte et des fiches réflexes destinées aux PME.

Références

- URLhaus, abuse.ch : <https://urlhaus.abuse.ch>
- Documentation scikit-learn : <https://scikit-learn.org/stable/>
- Documentation pandas : <https://pandas.pydata.org/docs/>
- Documentation Python : <https://docs.python.org/3/>
- PhiUSIIL Phishing URL Dataset, UCI Machine Learning Repository : <https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>
- Dépôt GitHub du projet : https://github.com/ALLAKORI/STAGE_PFA_CMRPI

A. Valeurs détaillées de l'exécution

Accuracy : 0.993172035030429
Precision : 0.9888464925154095
Recall : 0.9993325917686318
F1-score : 0.9940618891306753

Matrice de confusion :

```
[[19885  304]
 [   18 26952]]
```